Elasticsearch uses *dynamic mapping* to deduce the data types of fields when a document is indexed for the first time by looking at the field values and deriving these types. Each field has a specific data type defined: the `rating` field is declared to be a `float`, `release_year` a `date` type, and so on. Elasticsearch dynamically determined that the type of the `title` field is `text` based on its string value (`"Godfather"`). Because the field is stamped as a `text` type, all full-text-related queries can be performed on this field.

An individual field can also be composed of other fields, representing multiple data types. In addition to creating the `title` field as a `text` type, Elasticsearch did something extra: using the `fields` object, it created an additional `keyword` type for the `title` field, thus making `title` a *multi-typed* field. However, it must be accessed using `title.keyword`.

Multi-typed fields can be associated with multiple data types. In our example, by default, the `title` field is mapped to `text` as well as the `keyword` type. `keyword` fields are used for exact value searches. Fields with this data type are left untouched and don't go through an analysis phase: they are not tokenized, synonymized, or stemmed. See section 4.7 for more about multi-typed fields.

Analysis of keyword fields

Fields declared as the `keyword` data type use a special analyzer called `noop` (the no-operation analyzer), and `keyword` data is not touched during the indexing process. This keyword analyzer spits out the entire field as one big token.

By default, `keyword` fields are not normalized, as the `normalizer` property on the `keyword` type is set to `null`. But we can customize and enable `normalizer` on the `keyword` data type by setting filters such as `german_normalizer`, `uppercase`, and so on. Doing so indicates that we want the `keyword` field to undergo normalization before indexing.

The process by which Elasticsearch deduces a mapping from field values is called *dynamic mapping* (more in the next section). While dynamic mapping is intelligent and convenient, be aware that it can also get schema definitions wrong. Elasticsearch can only go so far when deriving a schema based on our document's field values. It may make incorrect assumptions, leading to erroneous index schemas that can produce incorrect search results.

We have learned that Elasticsearch derives and determines the data types of fields and creates a schema for us dynamically if the schema doesn't exist when the first document comes through for indexing. In the next section, we discuss how Elasticsearch determines types and the downsides of dynamic mapping.

4.2 Dynamic mapping

When we index a document for the first time, both the mapping and the index are created automatically. The engine doesn't complain if we don't provide the schema